# Software Requirements Specification (SRS)

## Student Project Collaboration and Experience Platform (TeamForge)

**Document Version:** 2.0
**Date:** October 17, 2026
**Status:** Approved Baseline

## Table of Contents


1. Preface
2. 1. Introduction
3. 1.1 Purpose
4. 1.2 Scope
5. 1.3 Definitions, Acronyms, and Abbreviations
6. 2. Overall Description & Operational Perspectives
7. 2.1 Product Perspective
8. 2.2 Operating Environment
9. 2.3 Design and Implementation Constraints
10. 2.4 Assumptions and Dependencies
11. 3. User Roles and Operational Modes
12. 3.1 Five Primary System Roles
13. 3.2 Dynamic Context-Based Role Switching
14. 3.3 Role-Permission Matrix
15. 4. Project Lifecycle & Collaboration Workflows
16. 4.1 Project Creation & Quota Enforcement
17. 4.2 Apply-Only Recruitment Workflow
18. 4.3 Member Removal & Voluntary Departure
19. 4.4 Project Completion & Publication Lock
20. 5. Task Tracking, Execution & Verification
21. 5.1 Action-Based Task Workflow
22. 5.2 Task Assignment Immutability Rules
23. 5.3 External Work Verification (URL + Notes)
24. 5.4 Project Progress Calculation
25. 6. Experience Points (XP) & Reputation System
26. 6.1 Base XP Rewards by Task Difficulty
27. 6.2 Idempotent Scoring & Transaction Ledger
28. 6.3 Reputation Rules & 30-Day Inactivity Decay
29. 6.4 XP Leaderboard Rankings (All-Time, Monthly, Weekly)
30. 7. Mentor Marketplace, Escrow & Earnings
31. 7.1 Mentor Eligibility & Verification
32. 7.2 Marketplace Directory & Mentor Profiles
33. 7.3 Canonical Escrow & Fee Breakdown Formula
34. 7.4 Session Lifecycle & Mentor Payout Settlement
35. 7.5 Locked Mentor Commitment Rule
36. 8. Mentor Recommendation Badges

37. 8.1 Award Criteria & Verification
38. 8.2 Reward Value & Cooldown Limitations
39. 8.3 Completed Project Badge Allowance
40. 9. Communication & Notification Infrastructure
    - 9.1 Workspace Real-Time Chat Board
    - 9.2 Event-Driven User Notification Center
    - 9.3 System Audit Logging
41. 10. Administrative Portal & Governance
    - 10.1 Portal Authentication & Permission Delegation
    - 10.2 User Moderation & Status Enforcement
    - 10.3 Project Supervision & Audit Inspection
    - 10.4 Revenue, Escrow Oversight & Payout Retry
42. 11. Platform Configuration & Maintenance Controls
    - 11.1 Super User Configuration Parameters
    - 11.2 System-Wide Maintenance Override (HTTP 503)
43. 12. External Interface Requirements
    - 12.1 User Interface Pages & Frontend Routes
    - 12.2 REST API Specification (Endpoints Table)
    - 12.3 Hardware & Software Interfaces
44. 13. Data Requirements & Entity Models
    - 13.1 Entity Schemas and Field Definitions
    - 13.2 State Machines & Lifecycle Enumerations
45. 14. Functional Requirements
    - 14.1 Authentication, Identity & Profiles (FR-01, FR-13, FR-14, FR-28)
    - 14.2 Project & Membership Management (FR-02, FR-03, FR-15, FR-16, FR-17)
    - 14.3 Task Execution & Progress (FR-04, FR-05, FR-06, FR-18)
    - 14.4 Gamification, Scoring & Leaderboards (FR-07, FR-08, FR-09, FR-10)
    - 14.5 Mentorship, Marketplace & Escrow (FR-11, FR-19, FR-20, FR-21, FR-22, FR-23)
    - 14.6 Communication, Notifications & Support (FR-24, FR-25, FR-29)
    - 14.7 Administrative Governance & Platform Control (FR-12, FR-26, FR-27, FR-30)
46. 15. Non-Functional Requirements
    - 15.1 Usability & Accessibility (NFR-01)
    - 15.2 Security & Access Control (NFR-02)
    - 15.3 Reliability, Idempotency & Fault Tolerance (NFR-03)
    - 15.4 Performance & Sliding-Window Rate Limiting (NFR-04)
    - 15.5 Data Integrity & In-Memory Storage Constraints (NFR-05)
47. 16. Business Rules Matrix
48. 17. System Architecture & Workflows (Mermaid Diagrams)
49. 18. Limitations and Out of Scope
50. 19. Appendices
    - Appendix A: Glossary of Terms
    - Appendix B: Requirement Mapping & Traceability Index

---

## Preface

This Software Requirements Specification (SRS) document provides the authoritative, complete, and audited technical specification for the **Student Project Collaboration and Experience Platform (TeamForge)**. It delineates the system's

operational architecture, actors, workflows, state models, algorithmic rules, administrative governance policies, and external REST API contracts as realized in the production implementation.

The platform is designed to help university students collaborate on real-world engineering projects, monitor task deliverables, verify external technical contributions, and build verifiable, transparent experience portfolios. Rather than hosting source code artifacts directly, TeamForge serves as an orchestration, coordination, and verification layer. Contributors submit verified external evidence (such as pull requests, commit hashes, or design documentation), which project owners review and approve.

A deterministic, rule-based gamification engine tracks productivity via **Experience Points (XP)** and dependability via a dynamic **Reputation Score**. The platform integrates an **Escrow-Backed Mentor Marketplace**, allowing experienced mentors to guide student teams through bonded commercial agreements, endorse high-performing contributors with **Recommendation Badges**, and receive automated fee payouts upon verified project completion. Comprehensive administrative oversight is maintained through multi-tiered role-based governance, immutable audit logging, and platform-wide configuration controls.

---

# 1. Introduction

## 1.1 Purpose

The purpose of TeamForge is to provide a structured, transparent, and trustworthy environment where students collaborate on practical software and engineering projects while accumulating verified records of their contributions. The platform enables project leads to define initiatives, decompose objectives into granular tasks, enforce accountability, and approve work evidence. Collaborators discover initiatives, submit proof of execution, and build cross-institutional credibility.

Additionally, the system bridges student initiatives with professional guidance through an integrated Mentor Marketplace. Built-in escrow mechanics safeguard financial commitments, ensuring that mentor payouts and platform fees are executed deterministically upon deliverable publication.

## 1.2 Scope

The scope of TeamForge encompasses: - **Account & Identity Management:** Public account registration, unique handle reservation, session lifecycle, and dynamic role switching between Collaborator, Project Owner, and Mentor. - **Project Coordination:** Creation of structured projects, capacity enforcement, task difficulty definition, and apply-only team recruitment. - **Deliverable Verification:** External hyperlink evidence submission, owner review/rejection workflows, and automated project progress tracking. - **Gamification Engine:** Difficulty-based XP yields (10/20/40 XP), task approval reputation bonuses (+8 Rep), 30-day inactivity decay, and multi-period leaderboards. - **Mentorship & Escrow Marketplace:** Mentor application verification, marketplace directory, 15% platform fee + 2% gateway fee escrow bonding, session management, and badge issuance. - **Communication & Notifications:** Chronological workspace team chat boards, event-driven user notifications, and immutable administrative audit logging. - **Administrative Governance:** Dedicated Super User and Admin portal, permission-based sectional access, user moderation (warning/suspension), telemetry tracking, and platform configuration override.

**Out of Scope:** The platform does not host Git repositories, execute CI/CD build pipelines, handle direct fiat currency banking rails, or store development source code on internal servers.

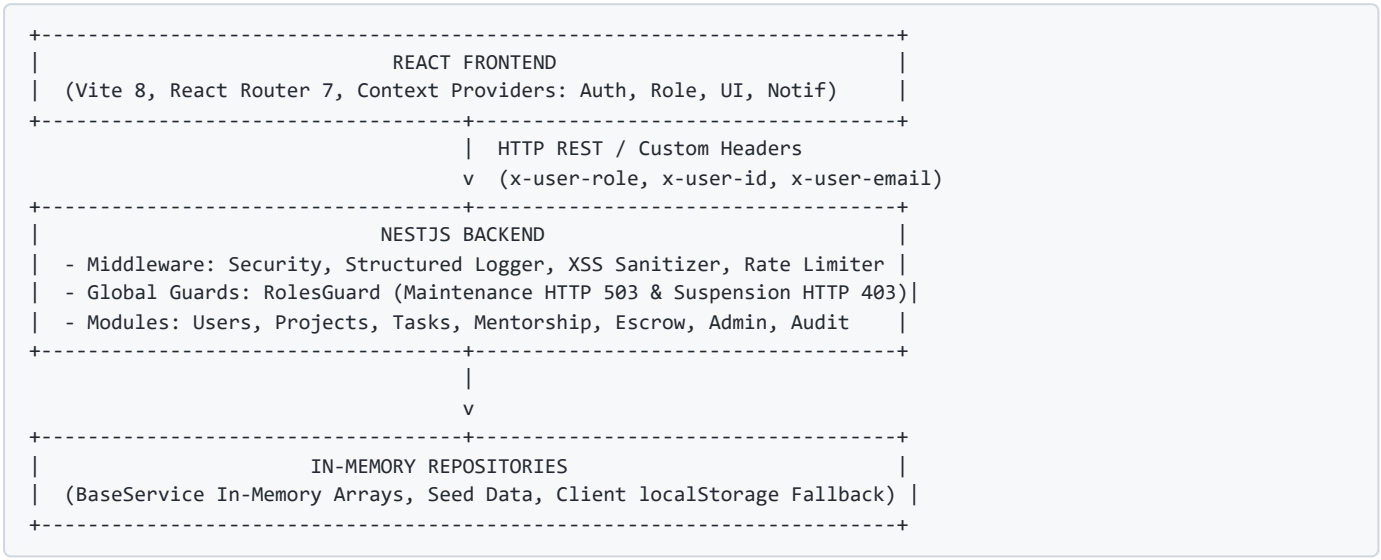
## 1.3 Definitions, Acronyms, and Abbreviations

- **API:** Application Programming Interface.
- **Collaborator:** A registered user participating in a project by executing assigned tasks.
- **DTO:** Data Transfer Object (NestJS payload validation model).
- **Escrow:** A contractual arrangement in which funds (session fee + platform fees) are held in trust until agreed project milestones are published.
- **Idempotency:** A property where an operation can be applied multiple times without changing the result beyond the initial application.
- **Mentor:** An expert vetted through an administrative application process who provides advisory support.
- **Project Owner:** The creator and administrative owner of a specific project workspace.
- **RBAC:** Role-Based Access Control.
- **Reputation Score:** A metric measuring contributor dependability, starting at 100 base points.
- **Super User:** The root platform administrator with exclusive access to config overrides and admin provisioning.
- **XP (Experience Points):** A non-decreasing numerical measure of total approved technical work completed on the platform.

## 2. Overall Description & Operational Perspectives

### 2.1 Product Perspective

TeamForge operates as a self-contained, multi-tier web application consisting of a React 19 single-page application (SPA) and a NestJS 12 REST API backend. The system acts as a coordination hub linking students, projects, and external code hosting services (e.g., GitHub, GitLab).




### 2.2 Operating Environment

- **Client Platforms:** Modern evergreen web browsers (Chromium >= 120, Firefox >= 120, Safari >= 17, Edge >= 120).
- **Frontend Runtime:** Node.js >= 20.x, Vite 8.3, React 19.2, React Router 7.18.
- **Backend Runtime:** Node.js >= 20.x, Express, NestJS 12.0, TypeScript 5.1.
- **Host System:** Cross-platform (Windows, Linux, macOS) exposing default HTTP port 3000 (Backend) and port 5173 (Frontend).

### 2.3 Design and Implementation Constraints

1. **Volatile In-Memory Storage:** The backend utilizes in-memory repository arrays. There is no persistent SQL or NoSQL database. Server restart resets state back to initial seed fixtures.
2. **Header-Based Authentication:** Identity is communicated via request headers ( `x-user-role` , `x-user-id` , `x-user-email` ). No cryptographic JWT signature verification is enforced on the API.
3. **Simulated Payment Processing:** Escrow financial bonding calculates fees and holds balances mathematically in memory without external payment gateway webhooks.
4. **Non-Interactive Background Task Execution:** Background operations (e.g., reputation decay cron) run on internal timer loops within the NestJS process.

### 2.4 Assumptions and Dependencies

- Users have valid external accounts (e.g., GitHub, GitLab, Figma, LinkedIn) to link as work evidence.
- Client browsers have JavaScript and Web Storage ( `localStorage` and `sessionStorage` ) enabled.
- Network infrastructure allows bidirectional HTTP REST traffic without stripping custom headers.

---

## 3. User Roles and Operational Modes

### 3.1 Five Primary System Roles

The system defines five distinct functional roles: 1. **Collaborator:** Default role assigned upon registration. Can explore projects, apply to join, start assigned tasks, submit external proof URLs, chat in workspaces, and earn XP. 2. **Project Owner:** Elevated role

assumed when creating an initiative. Defines tasks, reviews join requests, assigns work to existing members, approves/rejects submissions, removes members for cause, and publishes completed projects. 3. **Mentor:** Professional role unlocked only after administrative review of an application. Can list marketplace profiles, set session fees, accept project bookings, advise workspaces, and award recommendation badges. 4. **Administrator:** Governance role with credentials granting access to the Admin Portal. Moderates accounts, inspects projects, reviews mentor applications, monitors revenue/escrow, and inspects audit logs. 5. **Super User:** Root administrator possessing all permissions plus exclusive authorization to provision/deprovision admins, rotate keys, and modify platform-wide configuration settings.

### 3.2 Dynamic Context-Based Role Switching

User accounts represent unified identities rather than siloed account types. A single student account operates across different roles depending on context: - A user can act as a **Collaborator** in Project A while acting as a **Project Owner** in Project B. - Users switch active operational context via the UI role selector, updating `sessionStorage.getItem('teamforge.role')`. - The **Mentor** role remains strictly locked until an application is approved by an administrator ( `user.profile.mentorUnlocked === true` ).

### 3.3 Role-Permission Matrix

| Permission / Action | Collaborator | Project Owner | Mentor | Administrator | Super User |
|---|---|---|---|---|---|
| Browse Projects & Apply | **Yes** | **Yes** | Read-Only | **Yes** | **Yes** |
| Create New Project | No | **Yes** | No | **Yes** | **Yes** |
| Assign Tasks to Team | No | **Yes** | No | **Yes** | **Yes** |
| Start & Submit Assigned Task | **Yes** | **Yes** (if member) | No | No | No |
| Approve / Reject Task Deliverable | No | **Yes** | No | **Yes** | **Yes** |
| Complete & Publish Project Link | No | **Yes** | No | **Yes** | **Yes** |
| Remove Member (>= 10 char reason) | No | **Yes** | No | **Yes** | **Yes** |
| Voluntary Departure from Project | **Yes** | No | No | No | No |
| Book Mentor with Escrow | No | **Yes** | No | No | No |
| List Marketplace Profile & Fee | No | No | **Yes** | No | No |
| Issue Recommendation Badge | No | No | **Yes** | **Yes** | **Yes** |
| Send Workspace Chat Messages | **Yes** | **Yes** | **Yes** | **Yes** | **Yes** |
| Access Admin Portal Sections | No | No | No | **Permission-Based** | **Full Access** |
| Moderate Users (Warn, Suspend) | No | No | No | **Yes** | **Yes** |
| Superuser Admin & Config Management | No | No | No | No | **Yes** |

## 4. Project Lifecycle & Collaboration Workflows

### 4.1 Project Creation & Quota Enforcement

Any user in the Project Owner role can initiate a new project by defining the title, description, objectives, duration, difficulty, and required skills. - The project status strictly defaults to `Open`. - **Quota Rule:** The backend verifies that the owner's active project count does not exceed `maxProjectsPerUser` (platform default: 10). Exceeding this limit returns HTTP 400.

### 4.2 Apply-Only Recruitment Workflow

Team assembly is strictly apply-only: 1. Prospective collaborators submit a join request ( `POST /join-requests` ) specifying role and statement of intent. 2. Direct invitations, invite codes, and statuses `Invited` / `Declined` are completely unsupported. 3. The Project Owner reviews pending requests and either approves ( `Approved` ) or declines ( `Rejected` ). 4. Approving a request automatically appends the applicant to `project.collaborators` up to `maxCollaboratorsPerProject` (default 20). 5. Approved collaborators arrive with **no pre-assigned tasks**. The owner must assign tasks individually.

### 4.3 Member Removal & Voluntary Departure

- **Owner Removal:** The project owner may remove a collaborator via `DELETE /projects/:id/members/:memberId`. The owner must submit an explanation reason of **at least 10 characters**.
- All unapproved tasks assigned to that collaborator are automatically reset to `Unassigned` and status `To Do`.
- The removed user's ID is permanently recorded in `project.removedUsers`; re-applying to the project is blocked.
- **Voluntary Departure:** A collaborator may leave via `POST /projects/:id/leave`. Unapproved tasks reset to `Unassigned` and `To Do`.
- **Mentor Exemption:** Mentors are permanently locked once accepted; neither removal nor departure is permitted.

### 4.4 Project Completion & Publication Lock

A project is formally completed **only** when the owner publishes the final project link ( `POST /projects/:id/complete` ): - **Pre-Conditions:** All project tasks must be in status `Completed` (100% approved). If any task remains open, in progress, or in review, completion is rejected with HTTP 400. - **Deliverable Validation:** The owner must provide a valid URL starting with `http://` or `https://`. - **Permanent Lock:** Upon publication, `project.isCompleted = true`, `status = 'Completed'`, and `progress = 100`. The project becomes immutable (read-only) across frontend and backend. - **Escrow Settlement:** Publication automatically triggers the non-blocking release of bonded mentor escrow payouts. - **Badge Exception:** Awarding mentor recommendation badges is the sole operational action permitted on completed projects.

---

## 5. Task Tracking, Execution & Verification

### 5.1 Action-Based Task Workflow

TeamForge implements an explicit action-driven task lifecycle: $$\text{To Do (Open)} \xrightarrow{\text{Start}} \text{In Progress} \xrightarrow{\text{Submit Proof}} \text{In Review} \xrightarrow{\text{Approve / Reject}} \text{Completed (Approved) / To Do (Open)}$$ - Kanban drag-and-drop boards are excluded. Status transitions occur strictly via button triggers.

### 5.2 Task Assignment Immutability Rules

- Tasks can be assigned only to verified project members (owner, collaborators, or mentor).
- **Assignment Freeze:** A task's assignee can be modified or set to `Unassigned` **only while in status** `To Do` **/** `Open`. Once started ( `In Progress`, `In Review`, or `Completed` ), reassignment is rejected with HTTP 400.

### 5.3 External Work Verification (URL + Notes)

Collaborators submit deliverables by providing: - A mandatory external URL link ( `proofUrl` ) pointing to verifiable work (e.g., PR, commit, deployed site). - Optional submission notes detailing technical choices or review guidance. - Binary file attachments are intentionally omitted from task submissions.

### 5.4 Project Progress Calculation

Project progress is calculated deterministically through a single unified formula across the system: $$\text{Progress \%} = \begin{cases} 100 & \text{if project is locked or completed} \\ 0 & \text{if total tasks} = 0 \\ \operatorname{round}\left(\frac{\text{Completed Tasks}}{\text{Total Tasks}} \times 100\right) & \text{otherwise} \end{cases}$$

---

## 6. Experience Points (XP) & Reputation System

### 6.1 Base XP Rewards by Task Difficulty

XP reflects cumulative productive output and is non-decreasing. Upon task approval, XP is awarded strictly based on task priority/difficulty:

| Task Difficulty / Priority Level | Base XP Awarded | Reputation Awarded |
|---|---|---|
| **Low / Beginner** | **10 XP** | **+8** |

| Task Difficulty / Priority Level | Base XP Awarded | Reputation Awarded |
|---|---|---|
| **Medium / Intermediate** | **20 XP** | **+8** |
| **High / Advanced** | **40 XP** | **+8** |

### 6.2 Idempotent Scoring & Transaction Ledger

- XP and reputation are awarded **exactly once** per task via an atomic check on the `xpAwarded: boolean` flag.
- Every award generates an immutable entry in the gamification ledger ( `addLedgerEntry` ) recording transaction ID, user ID, type ( `XP` or `REPUTATION` ), amount, reason, and ISO timestamp.

### 6.3 Reputation Rules & 30-Day Inactivity Decay

- All accounts initialize with a baseline score of **100 Reputation**.
- Approving a task awards **+8 Reputation**.
- Receiving a Mentor Recommendation Badge awards **+8 Reputation** (and **+250 XP**).
- **Reputation Decay Cron:** To ensure reputation reflects current engagement, accounts with zero task activity for **30 consecutive days** incur a 2% decay: $$\text{Reputation}_{\text{new}} = \operatorname{round}\left(\text{Reputation} \times 0.98, 2\right)$$
- Negative deductions for revisions, missed deadlines, or admin warnings are omitted from code and designated out-of-scope.

### 6.4 XP Leaderboard Rankings (All-Time, Monthly, Weekly)

The platform publishes a competitive leaderboard ranking contributors strictly by XP: - Ties are supported (users with identical XP share the same rank index). - Three time filters are supported: `alltime` , `monthly` , and `weekly` . - Output columns: Rank, User Name, Initials Avatar, XP, Reputation, Completed Tasks, and Total Projects.

---

## 7. Mentor Marketplace, Escrow & Earnings

### 7.1 Mentor Eligibility & Verification

Users apply for mentor status ( `POST /mentor-applications` ) by submitting career history, LinkedIn profile URL, and years of experience. - The applicant's experience must meet or exceed `mentorMinYearsExperience` from platform config (default: **4 years**). Applications below this threshold are rejected with HTTP 400. - An Administrator evaluates credentials and approves the application ( `PUT /mentor-applications/:id/approve` ), unlocking the `Mentor` role.

### 7.2 Marketplace Directory & Mentor Profiles

Verified mentors manage public listings defining: - Professional headline, bio, skills tags, and experience years. - Session pricing (whole rupees, INR). - Session duration (default 60 minutes) and availability schedule ( `Anytime` , `Weekdays` , `Weekends` ). - Both Project Owners (browsing directory) and Mentors (profile preview) utilize the identical `MentorCard` component.

### 7.3 Canonical Escrow & Fee Breakdown Formula

Mentorship booking requires escrow funding by the project owner: $$\begin{aligned} \text{Base Price} &= P \\ \text{Platform Fee (15\%)} &= \operatorname{round}(P \times 0.15) \\ \text{Gateway Fee (2\%)} &= \operatorname{round}(P \times 0.02) \\ \text{Total Funded by Owner} &= P + \text{Platform Fee} + \text{Gateway Fee} \\ \text{Mentor Payout on Release} &= P \end{aligned}$$

*Example: For a session price of ₹1,000, Platform Fee = ₹150, Gateway Fee = ₹20. The owner funds ₹1,170 into escrow. Upon release, the mentor receives ₹1,000, and the platform retains ₹150 revenue.*

### 7.4 Session Lifecycle & Mentor Payout Settlement

- **Escrow State Flow:** `PaymentPending` $\to$ `Funded` $\to$ `Distributed` (Released) or `Refunded` .
- **Decline / Cancellation / Project Deletion:** The total amount funded by the owner (100% of price and all fees) is refunded in full.
- **Completion / Project Publication:** Publishing the project executes `releaseEscrowForProject` , releasing 100% of the session fee to the mentor.

- **Payout Retry:** If network or runtime issues cause an escrow release failure, the session transitions to `payout_pending`. Admins or mentors can trigger `POST /mentor-marketplace/sessions/:id/retry-payout`.

### 7.5 Locked Mentor Commitment Rule

To protect project stability, once a mentor accepts a booking request ( `status = 'active'` ), the mentor **cannot leave or be removed** from the project: `ForbiddenException: A mentor cannot leave or be removed once a project has started.`

---

## 8. Mentor Recommendation Badges

### 8.1 Award Criteria & Verification

Mentors endorse outstanding student contributors with verifiable badges: - **Authorization:** Only the assigned, approved mentor of that project (or an Administrator) can issue a badge. - **Recipient Eligibility:** The recipient must be an active project member (collaborator or project owner). Self-awarding is forbidden.

### 8.2 Reward Value & Cooldown Limitations

- Awarding a badge grants the recipient **+250 XP** and **+8 Reputation**.
- Sets `user.profile.hasMentorBadge = true` and appends badge metadata to `user.data.mentorRecommendations`.
- **Constraint:** A mentor may award a maximum of **one badge per contributor per project**. Duplicate awards are rejected with HTTP 400.

### 8.3 Completed Project Badge Allowance

Even after a project is locked and completed, issuing recommendation badges remains **unlocked** so mentors can conduct post-project evaluations.

---

## 9. Communication & Notification Infrastructure

### 9.1 Workspace Real-Time Chat Board

Each project workspace features a chronological team chat board: - Messages record `senderId`, `senderName`, `content`, `timestamp`, and `role`. - Roles are dynamically resolved to display strings: `Owner`, `Collaborator`, `Mentor`, `Admin`, or `Super User`. - Frontend polls for new messages every 3 seconds while on the chat tab.

### 9.2 Event-Driven User Notification Center

The system generates user-scoped notifications for key lifecycle events: - `TASK_ASSIGNED`, `TASK_SUBMITTED`, `TASK_APPROVED`, `TASK_REJECTED`. - `JOIN_REQUEST`, `APPLICATION_APPROVED`, `APPLICATION_REJECTED`. - `MENTOR_SESSION_BOOKED`, `MENTOR_SESSION_ACCEPTED`, `MENTOR_SESSION_DECLINED`, `MENTOR_SESSION_COMPLETED`. - `MENTOR_BADGE_AWARDED`, `PROJECT_COMPLETED`, `ACCOUNT_WARNED`, `ACCOUNT_SUSPENDED`. - Notifications support individual and bulk mark-as-read ( `PATCH /notifications/:id/read`, `POST /notifications/read-all` ).

### 9.3 System Audit Logging

The platform maintains an immutable audit log capturing significant events: - Records `timestamp`, `action`, `actor`, `target`, `entityType`, `entityId`, `event` description, and `details`. - Filterable and inspectable via the Admin Portal Audit Log viewer.

---

## 10. Administrative Portal & Governance

### 10.1 Portal Authentication & Permission Delegation

The Admin Portal operates via a separate login gateway ( `/admin/login` ): - Root Super User: `superuser@teamforge.io` (credential-authenticated). - Default Admin: `admin@teamforge.io` (credential-authenticated). - **Delegated Admins:** The Super User creates secondary admin accounts ( `/admin/su-admins` ) with granular section permissions: `dashboard`, `users`, `projects`,

`mentor_apps`, `revenue`, `audit`. - Exiting the portal clears administrative session storage and redirects to the public landing page (`/`).

### 10.2 User Moderation & Status Enforcement

Administrators moderate user accounts via dedicated actions: - **Warn User:** Issues an official warning with a mandatory reason, setting status to `Warned` and sending notification. - **Suspend User:** Sets status to `Suspended`. Intercepts subsequent user requests with **HTTP 403 Forbidden** and clears client sessions. - **Reactivate / Activate:** Restores status to `Active` and clears warning/suspension flags. - **Flag / Unflag:** Sets status to `Flagged` for monitoring without barring access.

### 10.3 Project Supervision & Audit Inspection

Administrators inspect global project listings, examine active deliverables, verify task completion statuses, and review system audit trails.

### 10.4 Revenue, Escrow Oversight & Payout Retry

The Admin Revenue and Escrow dashboard tracks: - Total sessions, awaiting responses, active mentorships, completed sessions, and refund volumes. - Total funds held in escrow, platform fee revenue (15%), and total paid out to mentors. - Administrative payout retry trigger for stuck sessions.

---

## 11. Platform Configuration & Maintenance Controls

### 11.1 Super User Configuration Parameters

Accessible strictly to Super Users (`/admin/su-config`), the platform configuration service controls:

| Parameter Key | Data Type | Default Value | Valid Range | Operational Purpose |
|---|---|---|---|---|
| `maxProjectsPerUser` | integer | `10` | 1 - 100 | Maximum active projects owned per user simultaneously |
| `maxCollaboratorsPerProject` | integer | `20` | 1 - 200 | Maximum team member capacity per project |
| `mentorMinYearsExperience` | integer | `4` | 0 - 50 | Minimum career tenure required for mentor approval |
| `xpPerTaskCompletion` | integer | `50` | 0 - 10,000 | Standard base XP parameter reference |
| `maintenanceMode` | boolean | `false` | true / false | System-wide traffic lockout toggle |
| `platformVersion` | string | `'1.4.2'` | SemVer | Active platform release version identifier |

### 11.2 System-Wide Maintenance Override (HTTP 503)

When `maintenanceMode` is toggled to `true`, the global NestJS `RolesGuard` intercepts all inbound HTTP requests: - Throws **HTTP 503 (Service Unavailable)** with message: "*Platform is undergoing maintenance. Please check back later.*" - **Exemptions:** Portal Admins, Super Users, and administrative endpoints (`/admin/*`, `/api/*`, `/uploads/*`) bypass maintenance mode to permit system maintenance.

---

## 12. External Interface Requirements

### 12.1 User Interface Pages & Frontend Routes

| URL Route | Component / Page | Access Control Guard | Primary Function |
|---|---|---|---|
| `/` | `LandingPage.jsx` | Public | Welcome portal, platform value proposition, call to action |
| `/login` | `Login.jsx` | Public | User authentication with email/username and password |

| URL Route | Component / Page | Access Control Guard | Primary Function |
|---|---|---|---|
| `/signup` | `Signup.jsx` | Public | Account registration (defaults to Collaborator role) |
| `/dashboard` | `Dashboard.jsx` | Authenticated | Overview of enrolled projects, assigned tasks, XP/Rep |
| `/projects` | `ProjectsMarketplace.jsx` | Authenticated | Browse active projects, filter by difficulty, submit join requests |
| `/applied-projects` | `AppliedProjects.jsx` | Authenticated | View submitted join requests and application statuses |
| `/my-work` | `MyWork.jsx` | Authenticated | List all projects where user is collaborator or owner |
| `/workspace/:projectId` | `CollaboratorWorkspace.jsx` | Authenticated (Member) | Collaborator workspace: overview, tasks, members, chat, submit proof |
| `/create-project` | `CreateProject.jsx` | Owner Protected | Project definition form (title, objectives, skills, difficulty) |
| `/my-projects` | `MyProjects.jsx` | Owner Protected | List projects owned by user with progress metrics |
| `/owner/mentors` | `MentorsDirectory.jsx` | Owner Protected | Browse mentor marketplace, filter rates, book with escrow |
| `/owner/workspace/:id` | `OwnedWorkspace.jsx` | Owner Protected | Owner workspace: assign tasks, approve deliverables, publish link |
| `/mentor/apply` | `MentorApplication.jsx` | Authenticated | Apply for mentor status with experience years and LinkedIn |
| `/mentor/projects` | `MentoredProjects.jsx` | Mentor Protected | List projects actively guided by mentor |
| `/mentor/requests` | `MentorRequests.jsx` | Mentor Protected | Review inbound mentorship booking requests (accept/decline) |
| `/mentor/earnings` | `MentorEarnings.jsx` | Mentor Protected | Track completed session earnings, held escrow, fee breakdowns |
| `/mentor/profile` | `MyMentorProfile.jsx` | Mentor Protected | Configure hourly rate, schedule, bio, and preview card |
| `/mentor/workspace/:id` | `MentorProjectWorkspace.jsx` | Mentor Protected | Mentor workspace: advice, tasks review, issue badges |
| `/help` | `Help.jsx` | Authenticated | Support center and ticket creation |
| `/leaderboard` | `Leaderboard.jsx` | Authenticated | Public contributor leaderboard (all-time, monthly, weekly) |
| `/notifications` | `Notifications.jsx` | Authenticated | View and mark notifications as read |
| `/profile` | `Profile.jsx` | Authenticated | User portfolio, badges, skills, and contribution history |
| `/settings` | `Settings.jsx` | Authenticated | Profile details, avatar upload, and Light/Dark theme switching |
| `/admin/login` | `AdminLogin.jsx` | Public (Portal) | Admin and Super User portal credential gateway |
| `/admin` | `AdminDashboard.jsx` | Admin Protected | Administrative KPIs, active users, project statistics |
| `/admin/users` | `AdminUsers.jsx` | Admin (users perm) | User account management, warnings, and suspensions |
| `/admin/users/:id` | `AdminUserDetails.jsx` | Admin (users perm) | Detailed user profile, history, contributions, and moderation |

| URL Route | Component / Page | Access Control Guard | Primary Function |
|---|---|---|---|
| `/admin/projects` | `AdminProjects.jsx` | Admin (projects perm) | Platform project directory inspection |
| `/admin/mentor-apps` | `AdminMentorApps.jsx` | Admin (mentor perm) | Review, approve, or reject mentor applications |
| `/admin/mentor-revenue` | `MentorRevenue.jsx` | Admin (revenue perm) | Escrow ledger oversight, revenue analytics, payout retry |
| `/admin/audit` | `AuditLog.jsx` | Admin (audit perm) | System-wide audit log inspector |
| `/admin/su-admins` | `SuperuserAdmins.jsx` | Super User Only | Provision and manage portal admins and sectional permissions |
| `/admin/su-config` | `SuperuserConfig.jsx` | Super User Only | Platform parameters and Maintenance Override controls |

### 12.2 REST API Specification (Endpoints Table)

| HTTP Method | API Path | Authorized Roles | Functional Purpose |
|---|---|---|---|
| `POST` | `/users` | Public | Register new user account with unique handle and email |
| `POST` | `/users/login` | Public | Authenticate user credentials, check suspension status |
| `GET` | `/users/username-available` | Public / Auth | Check username handle collision (case-insensitive) |
| `GET` | `/users/:id` | Authenticated | Retrieve user profile, skills, badges, and verified projects |
| `PATCH` | `/users/:id` | User / Admin | Update profile details (name, bio, phone, LinkedIn, skills) |
| `GET` | `/projects` | Authenticated | List all projects; supports status and completion filtering |
| `GET` | `/projects/:id` | Authenticated | Retrieve single project details, members, and progress |
| `POST` | `/projects` | Project Owner, Admin | Create project, enforce quota, default status to `Open` |
| `PATCH` | `/projects/:id` | Project Owner, Admin | Update project metadata (forbids direct member edits) |
| `POST` | `/projects/:id/complete` | Project Owner, Admin | Publish final URL, verify tasks, mark completed, lock |
| `POST` | `/projects/:id/leave` | Collaborator | Voluntarily leave project; resets assigned unapproved tasks |
| `DELETE` | `/projects/:id/members/:mId` | Project Owner, Admin | Remove member with >= 10 char reason; records in blacklist |
| `DELETE` | `/projects/:id` | Project Owner, Admin | Delete project; refunds escrow and notifies mentor |
| `GET` | `/tasks/project/:projectId` | Authenticated | List all tasks associated with a specific project |
| `POST` | `/tasks` | Owner, Collab, Admin | Create task with difficulty/priority and member assignee |
| `PATCH` | `/tasks/:id` | Owner, Collab, Admin | Update task; locks assignee once task is started |

| HTTP Method | API Path | Authorized Roles | Functional Purpose |
|---|---|---|---|
| `PATCH` | `/tasks/:id/status` | Owner, Collab, Admin | Update status (Start, Submit Proof with URL, Approve) |
| `DELETE` | `/tasks/:id` | Project Owner, Admin | Delete task from active project; updates progress |
| `GET` | `/join-requests` | Authenticated | List join requests filterable by project, user, or status |
| `POST` | `/join-requests` | Authenticated (User) | Submit join request to project (apply-only) |
| `PUT` | `/join-requests/:id` | Project Owner, Admin | Approve or reject join request; adds member to project |
| `GET` | `/mentor-applications` | Authenticated | List mentor applications (applicant-scoped or admin view) |
| `POST` | `/mentor-applications` | Authenticated | Submit mentor application; checks min years experience |
| `PUT` | `/mentor-applications/:id/approve` | Admin, Superuser | Approve application, assign Mentor role, unlock features |
| `PUT` | `/mentor-applications/:id/reject` | Admin, Superuser | Reject mentor application |
| `GET` | `/mentor-marketplace` | Authenticated | Browse verified mentor profiles, filter by price and skill |
| `POST` | `/mentor-marketplace/profile` | Mentor Only | Create or update mentor marketplace listing profile |
| `POST` | `/mentor-marketplace/book` | Project Owner | Book mentor, calculate fees, hold funds in escrow |
| `POST` | `/mentor-marketplace/sessions/:id/accept` | Mentor Only | Mentor accepts booking; assigns mentor to project |
| `POST` | `/mentor-marketplace/sessions/:id/decline` | Mentor Only | Mentor declines booking; refunds escrow in full |
| `GET` | `/mentor-marketplace/my-sessions` | Authenticated | Retrieve caller's booked or mentoring sessions |
| `GET` | `/mentor-marketplace/admin/stats` | Admin, Superuser | Administrative escrow and revenue statistics |
| `POST` | `/mentor-marketplace/sessions/:id/retry-payout` | Admin, Mentor | Retry release for session stuck in `payout_pending` |
| `POST` | `/mentorship/badge` | Mentor, Admin | Award recommendation badge to collaborator (+250 XP, +8 Rep) |
| `GET` | `/mentorship/badges/:userId` | Authenticated | Retrieve badges awarded to a specific user |
| `GET` | `/communication/messages/project/:id` | Member, Admin | Retrieve project workspace chat history chronologically |
| `POST` | `/communication/messages` | Member, Admin | Send message to project chat with sender name and role |
| `GET` | `/notifications` | Authenticated | Retrieve user notifications reverse-chronologically |
| `PATCH` | `/notifications/:id/read` | Authenticated | Mark single notification as read |
| `POST` | `/notifications/read-all` | Authenticated | Bulk mark all caller notifications as read |
| `GET` | `/leaderboard` | Authenticated | Retrieve rankings for `alltime`, `monthly`, or `weekly` |

| HTTP Method | API Path | Authorized Roles | Functional Purpose |
|---|---|---|---|
| `GET` | `/admin/stats` | Admin, Superuser | Global usage metrics, user states, project breakdown |
| `GET` | `/admin/users` | Admin, Superuser | List all users with moderation statuses and project counts |
| `GET` | `/admin/users/:id` | Admin, Superuser | Retrieve detailed user profile, warnings, and contributions |
| `POST` | `/admin/login` | Public (Portal) | Authenticate portal admin or superuser credentials |
| `PATCH` | `/admin/users/:id/status` | Admin, Superuser | Moderate user status: suspend, activate, warn, flag |
| `GET` | `/admin/audit` | Admin, Superuser | Retrieve immutable system audit log entries |
| `GET` | `/admin/config` | Admin, Superuser | Retrieve current platform configuration parameters |
| `PUT` | `/admin/config` | Super User Only | Update platform config (quotas, limits, maintenance) |
| `GET` | `/portal-admins` | Super User Only | List all secondary portal admin accounts |
| `POST` | `/portal-admins` | Super User Only | Provision new portal admin with granular permissions |
| `PUT` | `/portal-admins/:id` | Super User Only | Update portal admin details, permissions, or password |
| `DELETE` | `/portal-admins/:id` | Super User Only | Deprovision portal admin account |
| `POST` | `/support` | Authenticated | Submit help desk inquiry ticket |
| `GET` | `/support` | Authenticated / Admin | List support tickets (requester-scoped or admin view) |
| `PUT` | `/support/:id/status` | Admin, Superuser | Update ticket status ( `Open` , `In Progress` , `Resolved` ) |
| `POST` | `/uploads/avatar` | Authenticated | Upload user profile avatar image (max 2MB, images only) |

### 12.3 Hardware & Software Interfaces

- **Web Storage:** Uses browser `localStorage` and `sessionStorage` for client session persistence.
- **File System:** Serves static uploaded assets via `/uploads/` using Express static assets.
- **Network Interface:** HTTP/1.1 and HTTP/2 over standard TCP/IP.

---

## 13. Data Requirements & Entity Models

### 13.1 Entity Schemas and Field Definitions

**User Entity**

- `id` (string, UUID or numeric): Unique user identifier.
- `name` (string): Full display name.
- `email` (string): Unique email address.
- `username` (string): Unique handle (alphanumeric, dots, underscores, dashes, 3-30 chars).
- `role` (enum): `Collaborator` , `Project Owner` , `Mentor` , `Administrator` , `Super User` .
- `status` (enum): `Active` , `Suspended` , `Warned` , `Flagged` , `Banned` .

- `skills` (string array): Technical skill tags.
- `linkedIn` (string, optional): LinkedIn profile URL.
- `phone` (string, optional): Indian 10-digit phone number.
- `profile` (object): Extended metadata containing `xp`, `rep`, `tasksCount`, `mentorUnlocked`, `hasMentorBadge`, `avatarUrl`, `uni`, `joined`.
- `data` (object): User sub-collections (`projects`, `warnings`, `mentorRecommendations`).

**Project Entity**

- `id` (string): Unique project identifier.
- `title` (string): Project name.
- `description` (string): Overview and problem statement.
- `objectives` (string, optional): Specific milestones.
- `ownerId` (string): User ID of project creator.
- `owner` (string): Display name of project creator.
- `mentorId` (string, optional): Assigned mentor ID.
- `mentor` (string, optional): Assigned mentor display name.
- `status` (enum): `Open`, `In Progress`, `Completed`, `Cancelled`.
- `isCompleted` (boolean): Flag indicating project is finished and locked.
- `finalLink` / `finishedLink` (string, optional): Published deliverable URL.
- `completedAt` (ISO string, optional): Completion timestamp.
- `progress` (number, 0-100): Percentage of completed tasks.
- `collaborators` (array): Team members (`{ id, name, role }`).
- `removedUsers` (array): Blacklisted members (`{ userId, userName, reason, removedAt }`).

**Task Entity**

- `id` (string): Unique task identifier.
- `projectId` (string): Foreign key referencing Project.
- `title` (string): Task deliverable summary.
- `description` (string): Detailed requirements.
- `priority` / `difficulty` (enum): `Low`, `Medium`, `High`.
- `status` (enum): `To Do`, `In Progress`, `In Review`, `Completed`.
- `assigneeId` (string, optional): Assigned member user ID.
- `assignee` / `assigneeName` (string): Assigned member display name.
- `proofUrl` / `proofLink` (string, optional): External URL of submitted work.
- `notes` (string, optional): Submitter notes for reviewer.
- `xpReward` (number): XP yield upon approval (10, 20, or 40).
- `xpAwarded` (boolean): Idempotency flag preventing duplicate scoring.
- `createdAt` (ISO string): Creation timestamp.

**Join Request Entity**

- `id` (string): Unique request identifier.
- `projectId` (string): Targeted project ID.
- `userId` (string): Applying user ID.
- `userName` (string): Applying user display name.
- `role` (string): Requested role (`Collaborator`).
- `statement` / `message` (string): Intent statement.
- `status` (enum): `Pending`, `Approved`, `Rejected`, `Completed`, `Removed`, `Left`.

**Mentor Marketplace Profile Entity**

- `id` (string): Unique profile identifier.

- `userId` (string): Foreign key referencing verified mentor user.
- `name` (string): Mentor full name.
- `title` (string): Professional headline.
- `bio` (string): Mentorship bio.
- `skills` (string array): Technical specialties.
- `experienceYears` (number): Years of professional experience.
- `sessionPrice` (number): Hourly fee in INR.
- `sessionDuration` (number): Duration in minutes (default 60).
- `availability` (string): Schedule availability ( `Anytime` , `Weekdays` , `Weekends` ).
- `isAvailable` (boolean): Flag indicating if accepting new bookings.

**Mentor Session & Escrow Entities**

- `id` (string): Unique session identifier.
- `mentorId` (string): Mentor marketplace profile ID.
- `studentId` / `ownerId` (string): Project owner ID who booked session.
- `projectId` (string): Target project ID.
- `agreedPrice` (number): Base session fee in INR.
- `platformFee` (number): 15% platform commission.
- `gatewayFee` (number): 2% gateway processing cost.
- `totalAmount` (number): Total funded into escrow.
- `payout` (number): Net mentor payout upon release.
- `escrowId` (string): Foreign key referencing Escrow Account.
- `status` (enum): `escrow_funded` , `active` , `completed` , `refunded` , `cancelled` , `payout_pending` .

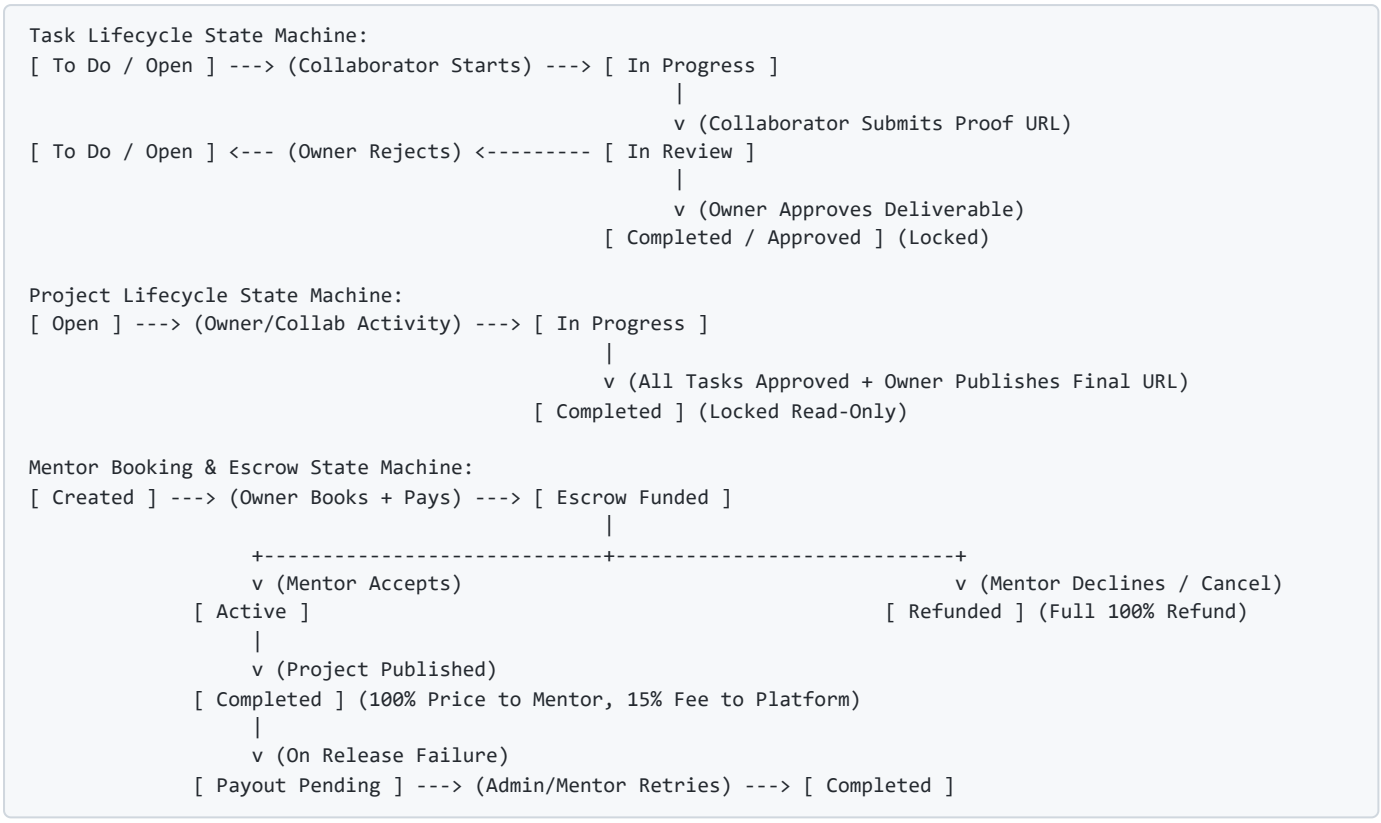
**Recommendation Badge Entity**

- `id` (string): Unique badge identifier.
- `recipientId` (string): Member receiving badge.
- `recipientName` (string): Member display name.
- `mentorId` (string): Issuing mentor user ID.
- `mentorName` (string): Issuing mentor display name.
- `projectId` (string): Project workspace reference.
- `projectTitle` (string): Project title.
- `badgeName` (string): Badge title (e.g., `Technical Excellence` ).
- `comment` (string): Mentor recommendation notes.
- `awardedAt` (ISO string): Issuance timestamp.

**Platform Config Entity**

- `maxProjectsPerUser` (number): Default 10.
- `maxCollaboratorsPerProject` (number): Default 20.
- `mentorMinYearsExperience` (number): Default 4.
- `xpPerTaskCompletion` (number): Default 50.
- `maintenanceMode` (boolean): Default false.
- `platformVersion` (string): Default `'1.4.2'` .

### 13.2 State Machines & Lifecycle Enumerations




---

## 14. Functional Requirements

### 14.1 Authentication, Identity & Profiles

#### FR-01: User Registration

- **Statement:** The system shall allow users to register an account by providing their full name, email address, password, desired username, technical skills, and optional LinkedIn URL.
- **Priority:** High
- **Acceptance Criteria:**
- Emails and usernames must be unique across the platform (case-insensitive).
- The system shall reject duplicate handles with HTTP 409 Conflict.
- Newly registered accounts shall be assigned the `Collaborator` role by default.
- Passwords must be securely accepted and stored.

#### FR-13: User Authentication & Session Verification

- **Statement:** The system shall authenticate users using either their registered email address or username handle combined with their password.
- **Priority:** High
- **Acceptance Criteria:**
- Valid credentials shall return the user entity and set client session state.
- If an account has status `Suspended` or `Banned`, login shall be denied with **HTTP 403 Forbidden** ("*This account is suspended. Contact support.*").
- Failed credentials shall return HTTP 401 Unauthorized or HTTP 404 Not Found.

#### FR-14: Multi-Role Operational Switching

- **Statement:** The system shall allow an authenticated user to dynamically switch their active operational role between `Collaborator` and `Project Owner`.

- **Priority:** High
- **Acceptance Criteria:**
- Switching roles updates client session state and transmits the corresponding `x-user-role` header on subsequent HTTP requests.
- Switching to the `Mentor` role shall be strictly prohibited unless the user has an approved mentor application on file (`mentorUnlocked === true`).

**FR-28: User Profile & Theme Settings**

- **Statement:** The system shall provide a user settings interface enabling users to update their profile information and customize appearance themes.
- **Priority:** Medium
- **Acceptance Criteria:**
- Users can update full name, unique handle, bio (10-180 chars), Indian phone number, LinkedIn link, skills tags, and upload an avatar image (max 2MB, JPEG/PNG/WebP).
- Users can select between `Light Theme` and `Dark Theme` by clicking theme cards, persisting preference in `localStorage` and applying CSS tokens.

---

## 14.2 Project & Membership Management

**FR-02: Project Creation & Platform Quota**

- **Statement:** The system shall allow users acting in the `Project Owner` or `Administrator` role to create new project initiatives.
- **Priority:** High
- **Acceptance Criteria:**
- Project creation requires specifying title, description, objectives, duration, difficulty, and required skills.
- Initial project status shall strictly default to `Open`.
- The system shall verify that the owner's active project count does not exceed `maxProjectsPerUser` (default 10); exceeding the quota returns HTTP 400.
- Direct invitation options are prohibited; initiatives operate exclusively on an apply-only basis.

**FR-03: Apply-Only Project Recruitment**

- **Statement:** The system shall allow collaborators to submit join requests to active projects, and allow project owners to review and decide on applications.
- **Priority:** High
- **Acceptance Criteria:**
- Collaborators submit an application via `POST /join-requests`.
- The system shall reject duplicate pending requests or applications from existing members.
- Project owners can approve (`Approved`) or reject (`Rejected`) requests.
- Approving an application adds the user to `project.collaborators` without pre-assigned tasks, subject to `maxCollaboratorsPerProject` (default 20).
- The system shall reject invitation creation (`Invited`) or decline workflows (`Declined`) with HTTP 400.

**FR-15: Project Completion & Deliverable Publication**

- **Statement:** The system shall permit the project owner to formally complete an active project by publishing a verified deliverable URL.
- **Priority:** High
- **Acceptance Criteria:**
- Completion requires 100% of tasks in status `Completed`. If unapproved tasks exist, completion is rejected with HTTP 400.
- The owner must supply a valid URL starting with `http://` or `https://`.
- Upon successful publication, `isCompleted` is set to true, status is set to `Completed`, progress is set to 100, and the project is permanently locked against further task or member edits.
- Completion automatically triggers release of bonded mentor escrow payouts.

**FR-16: Project Member Removal for Cause**

- **Statement:** The system shall allow project owners to remove collaborators from an active project by submitting a mandatory justification reason.
- **Priority:** Medium
- **Acceptance Criteria:**
- The removal reason must contain at least **10 characters**.
- Unapproved tasks assigned to the removed member are automatically reset to `Unassigned` and `To Do`.
- The member's ID is recorded in `project.removedUsers`; future join requests from that user are blocked.
- Mentors cannot be removed once accepted.

**FR-17: Collaborator Voluntary Departure**

- **Statement:** The system shall allow collaborators to voluntarily leave an active project workspace.
- **Priority:** Medium
- **Acceptance Criteria:**
- Departing resets all unapproved tasks assigned to that collaborator to `Unassigned` and `To Do`.
- Completed tasks and earned XP/reputation remain on the user's profile.
- Project owners and mentors are forbidden from using the leave action.

---

## 14.3 Task Execution & Progress

**FR-04: Task Definition & Assignment Freezing**

- **Statement:** The system shall allow project owners to define project tasks with difficulty ratings and assign them to existing project members.
- **Priority:** High
- **Acceptance Criteria:**
- Tasks require title, description, priority/difficulty ( `Low`, `Medium`, `High` ), and an optional assignee from existing team members.
- Base XP is calculated automatically: Low = 10, Medium = 20, High = 40.
- The assignee can be changed or cleared **only while status is** `To Do` **/** `Open`. Reassigning tasks that are `In Progress`, `In Review`, or `Completed` is rejected with HTTP 400.
- Task creation on completed/locked projects is rejected with HTTP 400.

**FR-05: Task Execution & Deliverable Proof Submission**

- **Statement:** The system shall allow the assigned collaborator to start a task and submit external hyperlink evidence upon completion.
- **Priority:** High
- **Acceptance Criteria:**
- Collaborators transition a task to `In Progress` using the start action.
- Deliverable submission requires an external URL ( `proofUrl` ) starting with `http://` or `https://` and optional notes.
- Submitting proof transitions task status to `In Review` and notifies the project owner.
- File uploads for task proof are unsupported.

**FR-06: Task Deliverable Review & Approval**

- **Statement:** The system shall allow the project owner to evaluate submitted task deliverables and either approve or request revisions.
- **Priority:** High
- **Acceptance Criteria:**
- Only the Project Owner (or Admin) can approve or reject deliverables.
- Approving sets status to `Completed`, awards base XP and +8 Reputation to the assignee, and sets `xpAwarded = true`.
- Rejecting transitions status back to `To Do` / `Open` for revision and notifies the collaborator.

**FR-18: Unified Progress Tracking**

- **Statement:** The system shall recalculate and persist project progress whenever task statuses transition.
- **Priority:** High
- **Acceptance Criteria:**
- Progress percentage is calculated as `Math.round((approvedTasks / totalTasks) * 100)`.
- Locked or completed projects evaluate to 100%.

---

## 14.4 Gamification, Scoring & Leaderboards

**FR-07: Experience Point Allocation**

- **Statement:** The system shall calculate and award Experience Points (XP) upon task approval based strictly on task difficulty.
- **Priority:** High
- **Acceptance Criteria:**
- Awards: Low = 10 XP, Medium = 20 XP, High = 40 XP.
- XP is awarded once per task using an idempotent `xpAwarded: true` check.
- XP totals never decrease once earned.

**FR-08: Reputation Scoring & Inactivity Decay**

- **Statement:** The system shall maintain contributor reputation scores reflecting positive contributions and activity recency.
- **Priority:** High
- **Acceptance Criteria:**
- All users start with 100 base reputation.
- Task approval awards +8 reputation; badge award awards +8 reputation.
- Users inactive for >30 days incur a 2% decay ( `rep = Math.round(rep * 0.98 * 100) / 100` ).

**FR-09: Contribution Portfolio Reporting**

- **Statement:** The system shall aggregate and display verified contributions on user profile and dashboard interfaces.
- **Priority:** Medium
- **Acceptance Criteria:**
- Displays list of completed projects, task deliverables, external proof URLs, and completion dates.

**FR-10: Multi-Period Leaderboard Generation**

- **Statement:** The system shall rank platform contributors based on accumulated XP across multiple time horizons.
- **Priority:** Medium
- **Acceptance Criteria:**
- Supports filters for `alltime`, `monthly`, and `weekly`.
- Ranks users descending by XP, handles ties correctly, and displays rank, user name, initials, XP, reputation, tasks, and projects.

---

## 14.5 Mentorship, Marketplace & Escrow

**FR-11: Mentor Recommendation Badges**

- **Statement:** The system shall allow an assigned project mentor to issue recommendation badges to contributing team members.
- **Priority:** High
- **Acceptance Criteria:**
- Only the project's assigned mentor can award badges.
- Self-awarding is prohibited.
- Maximum of one badge per mentor per contributor per project.
- Awarding grants +250 XP and +8 Reputation, sets `hasMentorBadge = true`, and logs in profile.
- Badge awarding is permitted on completed/locked projects.

**FR-19: Mentor Application & Experience Verification**

- **Statement:** The system shall provide an application process for users seeking mentor status, enforcing minimum career tenure.
- **Priority:** High
- **Acceptance Criteria:**
- Applicants submit years of experience, LinkedIn URL, skills, and motivation.
- Experience years must satisfy `mentorMinYearsExperience` (platform config, default 4). Applications with fewer years are rejected with HTTP 400.
- Admin approval unlocks the Mentor role and permissions.

**FR-20: Mentor Marketplace Profile Management**

- **Statement:** The system shall allow approved mentors to publish and manage a marketplace listing profile.
- **Priority:** High
- **Acceptance Criteria:**
- Mentors specify hourly session fee (in INR), bio, skills, experience, and availability schedule ( `Anytime` , `Weekdays` , `Weekends` ).
- Session fee must be greater than 0 INR.

**FR-21: Escrow-Backed Mentorship Booking**

- **Statement:** The system shall enable project owners to book mentors for active projects through an automated escrow hold.
- **Priority:** High
- **Acceptance Criteria:**
- Financial breakdown: Owner pays `sessionPrice + 15% platformFee + 2% gatewayFee` .
- Total amount is locked in escrow ( `EscrowState.Funded` ).
- Mentor decline or cancellation refunds the full amount paid to the owner.
- Mentor acceptance locks the mentor into the project ( `status = 'active'` ).

**FR-22: Mentorship Escrow Settlement on Project Publication**

- **Statement:** The system shall automatically release escrowed mentor funds upon verified project completion.
- **Priority:** High
- **Acceptance Criteria:**
- Publishing the project triggers `releaseEscrowForProject` .
- The mentor receives 100% of agreed `sessionPrice` , and the platform retains the 15% platform fee.
- Release failures transition session to `payout_pending` without blocking project completion.

**FR-23: Mentor Earnings & Admin Revenue Auditing**

- **Statement:** The system shall provide dedicated interfaces for mentors to track earnings and for administrators to audit platform escrow and revenue.
- **Priority:** Medium
- **Acceptance Criteria:**
- Mentors view completed sessions, released payouts, and funds in escrow.
- Admins inspect total volume, platform fee revenue, active holds, and can trigger payout retries.

---

## 14.6 Communication, Notifications & Support

**FR-24: Project Workspace Team Chat**

- **Statement:** The system shall provide an internal project workspace chat board displaying chronological communication.
- **Priority:** High
- **Acceptance Criteria:**
- Messages display sender ID, full name, resolved role string ( `Owner` , `Collaborator` , `Mentor` , `Admin` ), text content, and timestamp.
- Access is restricted to project team members and administrators.

**FR-25: Event-Driven User Notifications**

- **Statement:** The system shall generate user-scoped notifications for platform events and support read tracking.
- **Priority:** High
- **Acceptance Criteria:**
- Notifications triggered for task assignments, submissions, approvals, rejections, join requests, bookings, badges, and moderation actions.
- Supports individual and bulk mark-as-read.

**FR-29: Help Desk & Support Ticketing**

- **Statement:** The system shall allow users to submit support inquiries and allow administrators to manage ticket resolution.
- **Priority:** Low
- **Acceptance Criteria:**
- Users submit tickets with subject, message, and category.
- Admins review and update status ( `Open` , `In Progress` , `Resolved` , `Closed` ).

---

## 14.7 Administrative Governance & Platform Control

**FR-12: Multi-Tiered Administrative Governance Portal**

- **Statement:** The system shall provide a dedicated administrative portal with permission-based sectional access and moderation controls.
- **Priority:** High
- **Acceptance Criteria:**
- Portal access requires admin or superuser credentials via `/admin/login` .
- Sections are guarded by permissions ( `dashboard` , `users` , `projects` , `mentor_apps` , `revenue` , `audit` ).
- User moderation actions include `warn` (with reason), `suspend` (blocks login/API with HTTP 403), `activate` , `flag` , and `unflag` .
- Exiting the portal clears administrative session tokens and returns to the landing page ( `/` ).

**FR-26: Platform Configuration Management**

- **Statement:** The system shall allow Super Users to view and modify global platform configuration parameters.
- **Priority:** High
- **Acceptance Criteria:**
- Super Users can modify `maxProjectsPerUser` (1-100), `maxCollaboratorsPerProject` (1-200), `mentorMinYearsExperience` (0-50), and `maintenanceMode` .
- Updates are validated, persisted in memory, and recorded in the audit log.
- "Provision New Accounts" option is excluded from platform config.

**FR-27: Maintenance Override Mode Enforcement**

- **Statement:** The system shall intercept non-administrative traffic when Maintenance Override is active.
- **Priority:** High
- **Acceptance Criteria:**
- Enabling `maintenanceMode` causes `RolesGuard` to return **HTTP 503 Service Unavailable** for non-admin requests.
- Portal Admins, Super Users, and administrative routes ( `/admin/*` , `/api/*` , `/uploads/*` ) bypass maintenance mode.

**FR-30: Portal Admin Delegation & Access Provisioning**

- **Statement:** The system shall allow Super Users to provision, configure, and deprovision secondary portal admin accounts.
- **Priority:** High
- **Acceptance Criteria:**
- Super Users specify display identity, email, strong password (8+ chars, upper, lower, digit, special), and assign specific module permissions.
- Super Users can rotate passwords, toggle status, and delete admin accounts.

## 15. Non-Functional Requirements

### 15.1 Usability & Accessibility (NFR-01)

- **Statement:** The user interface shall provide an intuitive, responsive experience supporting Light and Dark theme modes, clear visual status indicators, and accessible form controls.
- **Acceptance Criteria:**
- Theme switching updates all CSS color tokens seamlessly without page reload.
- Project progress is visually indicated via progress bars and state badges.
- Destructive actions (member removal, leaving project, suspension) require explicit confirmation dialogs.

### 15.2 Security & Access Control (NFR-02)

- **Statement:** The system shall enforce role-based access control and input sanitization across all endpoints.
- **Acceptance Criteria:**
- All mutating requests (POST, PATCH, PUT) pass through the `SanitizerMiddleware` to strip dangerous HTML and XSS injection vectors.
- Role authorization is enforced via NestJS `RolesGuard`. Missing role headers return HTTP 401 Unauthorized; insufficient roles return HTTP 403 Forbidden.
- Suspended accounts are intercepted with HTTP 403 across all authenticated endpoints.

### 15.3 Reliability, Idempotency & Fault Tolerance (NFR-03)

- **Statement:** Financial escrow state transitions and gamification scoring transactions shall execute idempotently and atomically.
- **Acceptance Criteria:**
- Approving an already-approved task shall not award duplicate XP or reputation.
- Escrow release or refund shall only be permitted from valid states ( `Funded` or `Disputed` ).
- Failed mentor escrow release during project publication shall transition to `payout_pending` without crashing or reverting project completion.

### 15.4 Performance & Sliding-Window Rate Limiting (NFR-04)

- **Statement:** The system shall protect backend services from denial-of-service and abuse via client-aware rate limiting.
- **Acceptance Criteria:**
- Global `RateLimiterMiddleware` enforces a limit of **5,000 requests per 1-hour sliding window per client** (keyed by `x-user-id` or IP).
- Exceeding the quota returns **HTTP 429 Too Many Requests**, accompanied by `Retry-After`, `X-RateLimit-Limit`, and `X-RateLimit-Reset` headers.
- Documentation ( `/api` , `/swagger` ) and health check ( `/ping` , `/health` ) endpoints are exempt.

### 15.5 Data Integrity & In-Memory Storage Constraints (NFR-05)

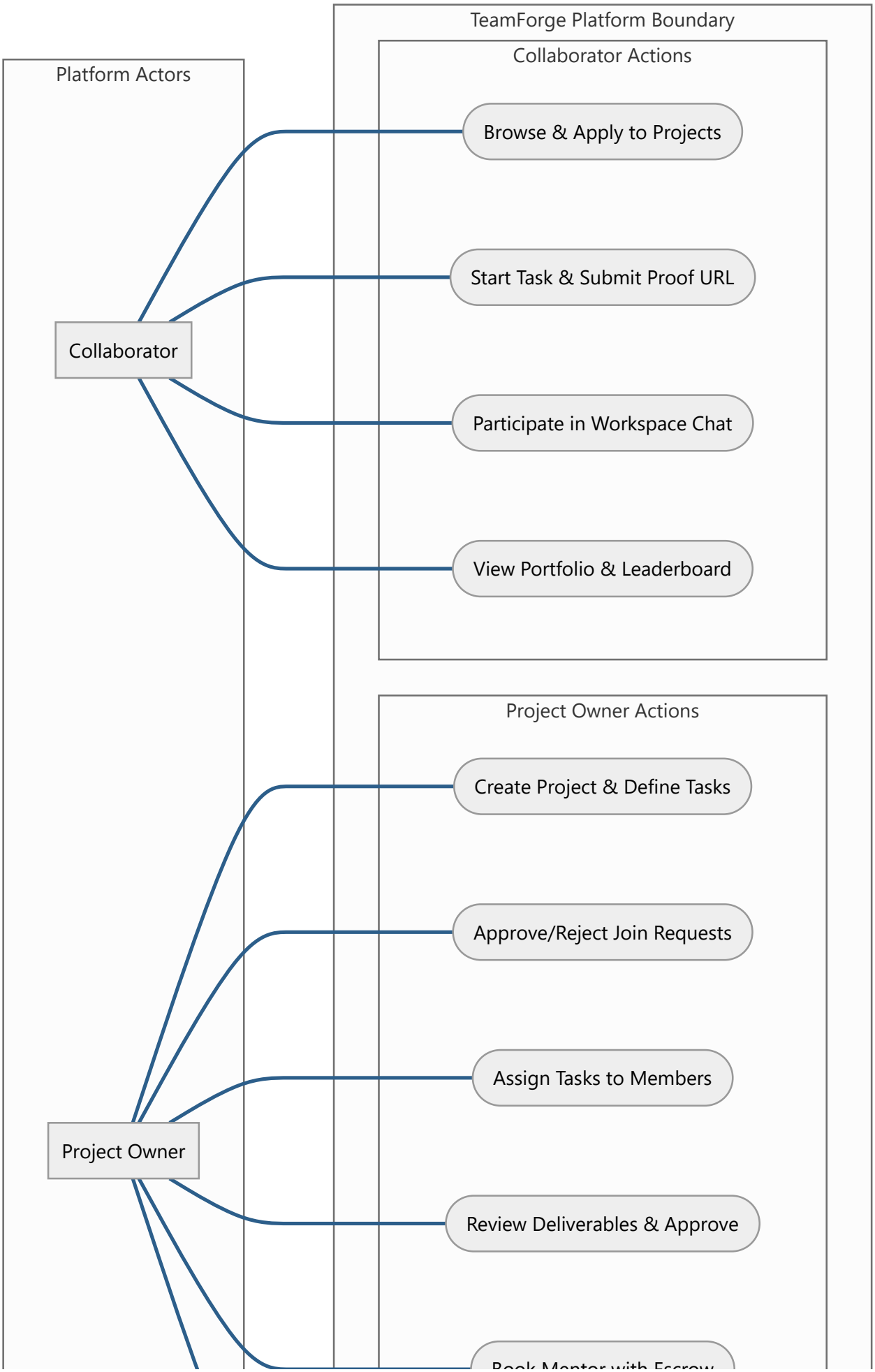
- **Statement:** The system architecture shall operate within the constraints of in-memory data structures while maintaining runtime referential integrity.
- **Acceptance Criteria:**
- All domain relationships (user-to-project, project-to-task, session-to-escrow) must maintain referential consistency within memory arrays.
- Client-side storage ( `localStorage` ) must be scoped by user ID to prevent cross-account data leaks during local session switching.

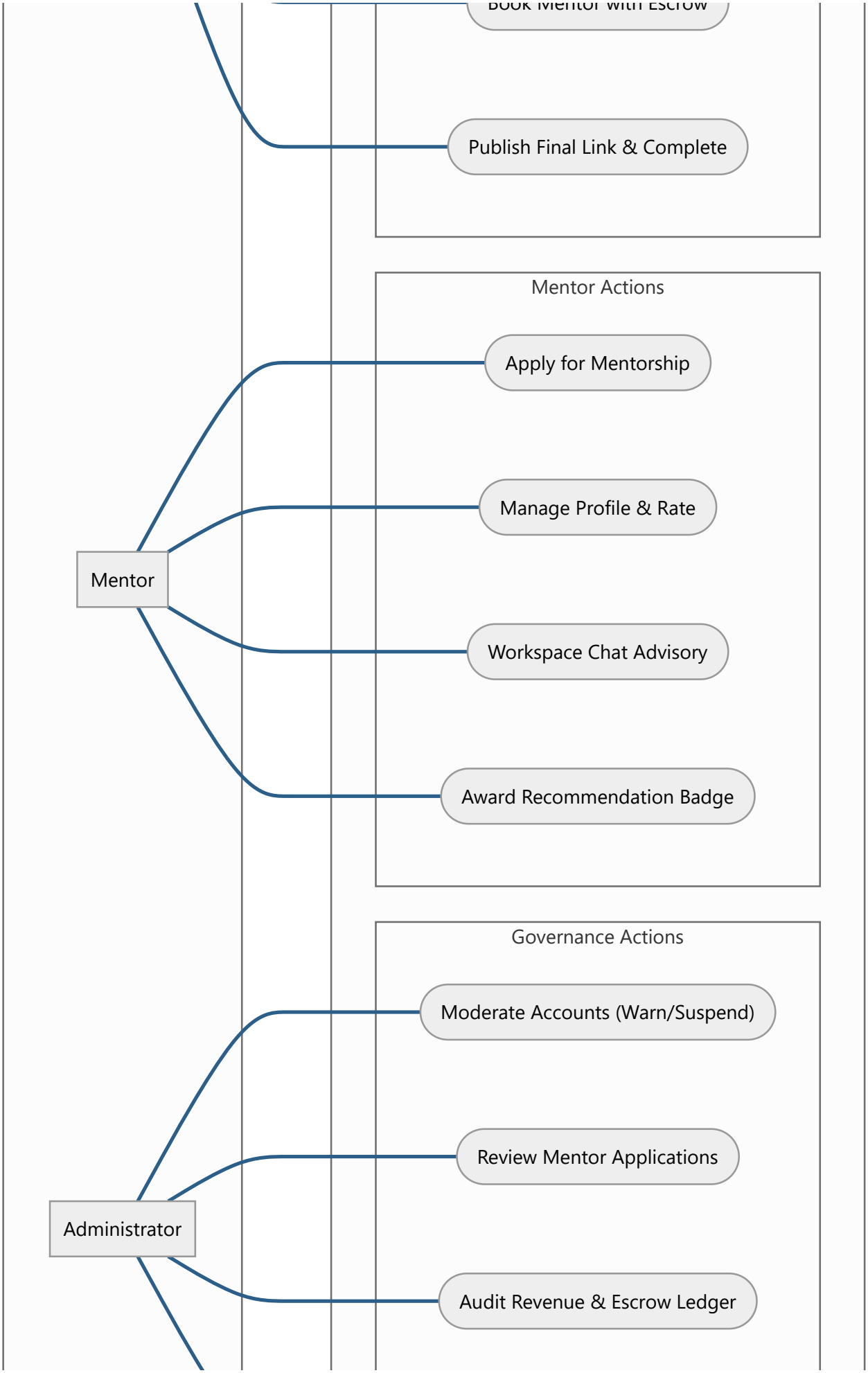
## 16. Business Rules Matrix

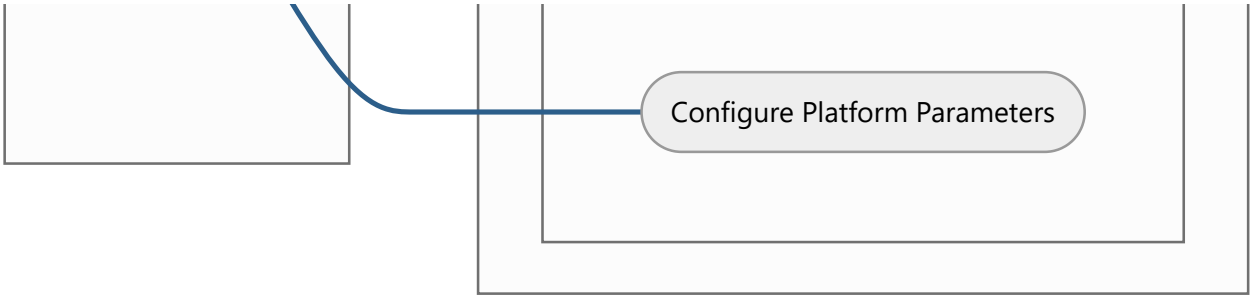
| Rule ID | Domain | Business Rule Statement | Enforcement Location |
|---|---|---|---|
| **BR-01** | Projects | A newly created project must always initialize with status `Open`. | `projects.repository.ts`, `projects.service.ts` |
| **BR-02** | Projects | Active projects owned by a single user cannot exceed `maxProjectsPerUser` (default 10). | `projects.service.ts` (`create`) |
| **BR-03** | Projects | Team member count cannot exceed `maxCollaboratorsPerProject` (default 20). | `projects.service.ts` (`addCollaborator`) |
| **BR-04** | Tasks | Task assignment is restricted exclusively to existing project team members. | `tasks.service.ts` (`validateTaskAssignee`) |
| **BR-05** | Tasks | A task's assignee cannot be modified once the task has started (`In Progress`, `In Review`, `Completed`). | `tasks.service.ts` (`update`) |
| **BR-06** | Tasks | Only the Project Owner (or Admin) can approve or reject task deliverables; collaborators cannot self-approve. | `tasks.service.ts` (`update`) |
| **BR-07** | Lifecycle | A project is completed ONLY when the owner publishes a valid URL and 100% of tasks are approved. | `projects.service.ts` (`completeProject`) |
| **BR-08** | Lifecycle | A completed project is permanently locked against task edits, task creation, or member removal. | `projects.service.ts` (`isLocked`) |
| **BR-09** | Mentorship | A mentor cannot leave or be removed once a project has started and booking is accepted. | `mentor-requests.service.ts`, `mentor-marketplace.service.ts`, `projects.service.ts` |
| **BR-10** | Escrow | Mentorship booking escrow holds `sessionPrice + 15% platformFee + 2% gatewayFee`. | `mentor-marketplace.service.ts` (`calculateEscrowBreakdown`) |
| **BR-11** | Escrow | Declining/cancelling mentorship refunds 100% of funded amount (price + fees) to owner. | `escrow.service.ts`, `mentor-marketplace.service.ts` |
| **BR-12** | Escrow | Publishing a project releases 100% of agreed session fee to mentor; platform retains 15% fee. | `mentor-marketplace.service.ts` (`completeSession`) |
| **BR-13** | Badges | Only assigned project mentor can award badges; max 1 badge per member per project; grants +250 XP & +8 Rep. | `mentorship.service.ts` (`issueBadge`) |
| **BR-14** | Governance | Suspended accounts are barred from login and intercepted with HTTP 403 on all API endpoints. | `roles.guard.ts`, `users.service.ts` (`login`) |
| **BR-15** | Governance | When Maintenance Mode is active, all non-admin traffic is blocked with HTTP 503. | `roles.guard.ts` (`canActivate`) |
| **BR-16** | Progress | Progress = approved tasks divided by total tasks (or 100% if locked/completed). | `progress.util.ts`, `progress.js` |
| **BR-17** | Removal | Removing a member requires >= 10 characters explanation and resets unapproved tasks to Open. | `projects.service.ts` (`removeMember`) |
| **BR-18** | Scoring | XP is awarded exactly once per task upon approval: Low = 10, Medium = 20, High = 40. | `tasks.constants.ts`, `tasks.service.ts` |

## 17. System Architecture & Workflows (Mermaid Diagrams)

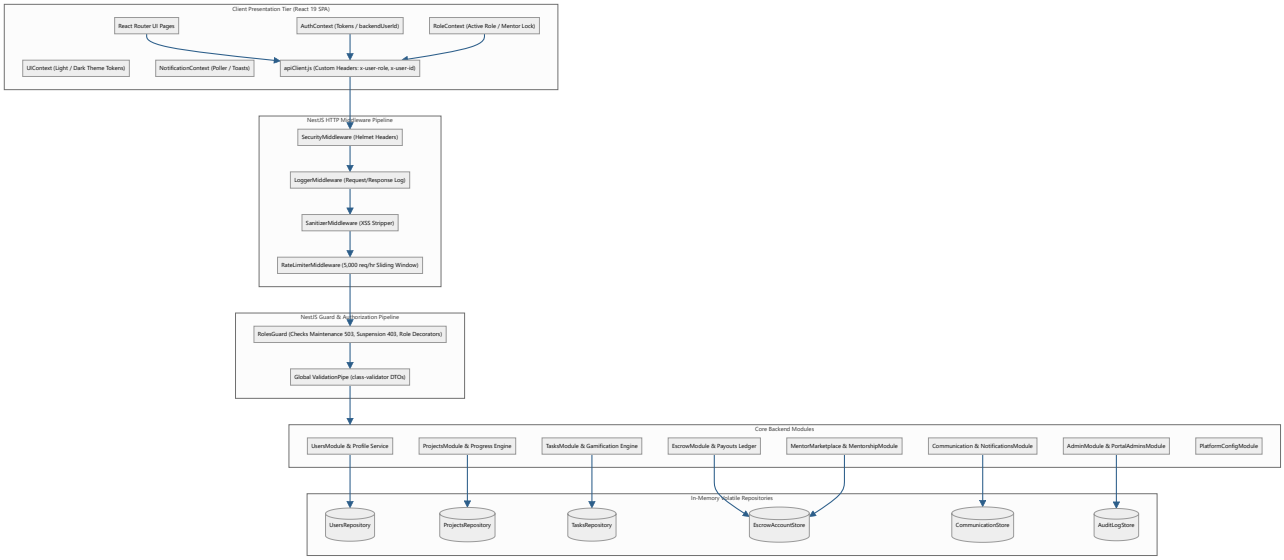
## Diagram 1: Role-Based System Use Case Overview


TeamForge Platform Boundary
Platform Actors
Collaborator Actions
Browse & Apply to Projects
Start Task & Submit Proof URL
Collaborator
Participate in Workspace Chat
View Portfolio & Leaderboard
Project Owner Actions
Create Project & Define Tasks
Approve/Reject Join Requests
Assign Tasks to Members
Project Owner
Review Deliverables & Approve

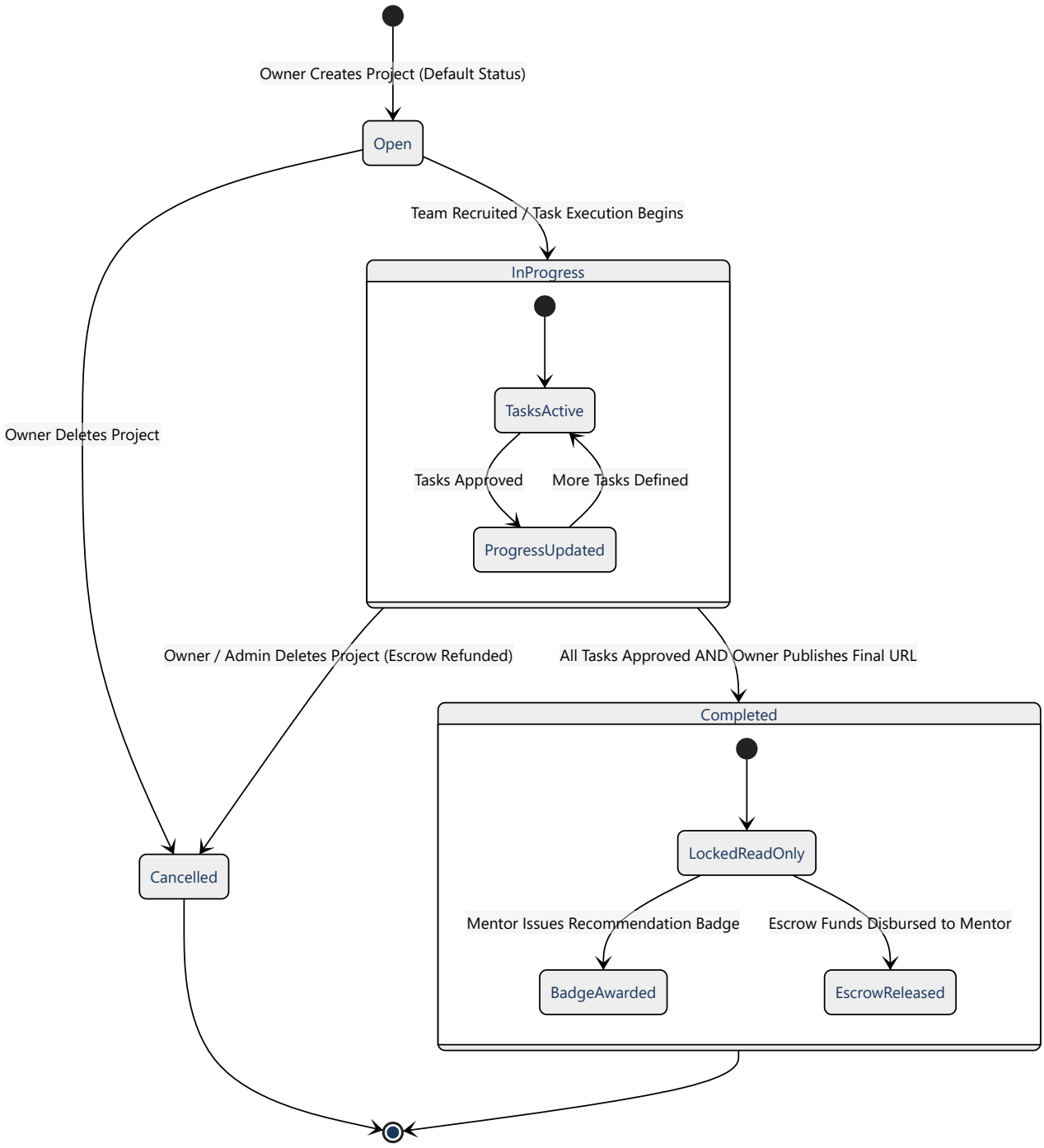

Book Mentor with Escrow
Publish Final Link & Complete
Mentor Actions
Apply for Mentorship
Manage Profile & Rate
Mentor
Workspace Chat Advisory
Award Recommendation Badge
Governance Actions
Moderate Accounts (Warn/Suspend)
Review Mentor Applications
Administrator
Audit Revenue & Escrow Ledger


Configure Platform Parameters

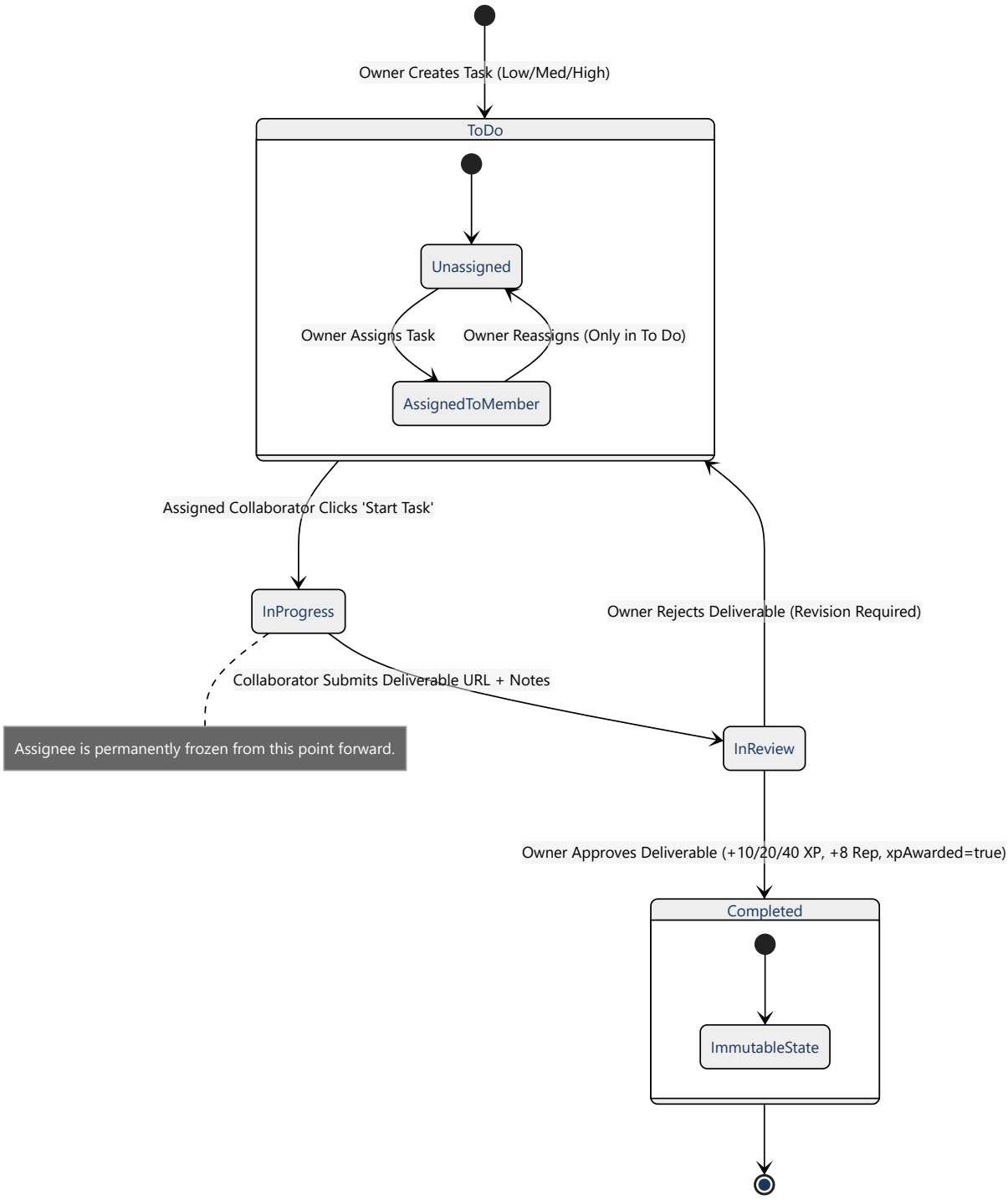
## Diagram 2: High-Level System Architecture



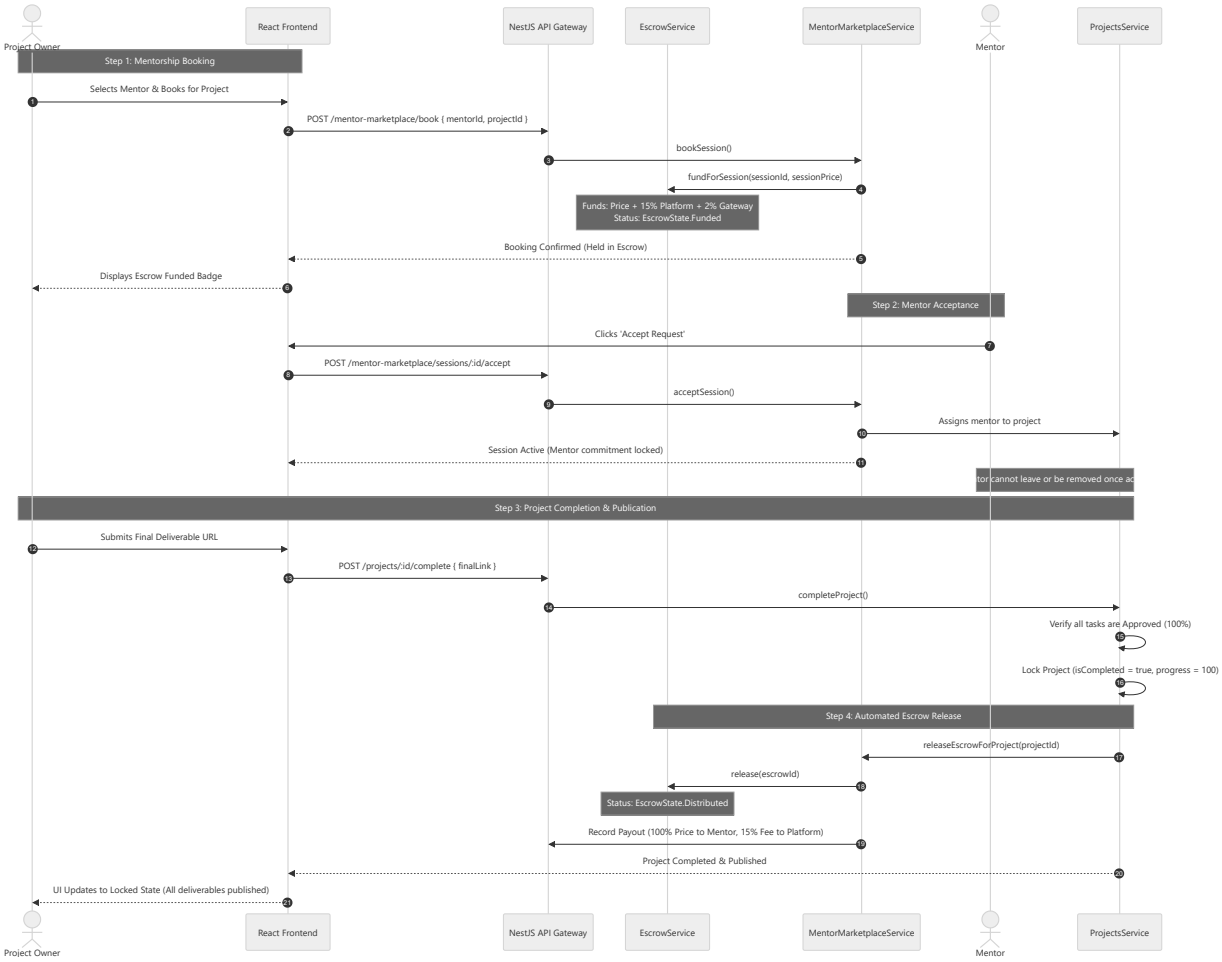
## Diagram 3: Project Lifecycle State Diagram



## Diagram 4: Task Lifecycle State Diagram



## Diagram 5: Mentor Booking, Escrow & Project Completion Sequence Diagram



## 18. Limitations and Out of Scope

To ensure complete transparency regarding the system's operational boundaries, the following architectural items are explicitly classified as **Limitations and Out of Scope**:

1. **Non-Persistent In-Memory Storage:** The backend does not connect to a relational (e.g., PostgreSQL, MySQL) or document (e.g., MongoDB) database. All entities reside in volatile memory arrays on the Node.js server. Restarting the server process reverts the database to the initial static seed data.
2. **Tokenless Header-Based Authentication:** The backend does not issue or cryptographically verify JSON Web Tokens (JWT) or secure session cookies. It relies on the client transmitting `x-user-role`, `x-user-id`, and `x-user-email` HTTP headers. Role validation occurs at the routing guard level.
3. **Simulated Commercial Payment Gateway:** Although fee calculations (15% platform cut, 2% gateway processing fee) and ledger balances are accurately tracked, no live banking API or payment gateway (e.g., Stripe, Razorpay) is integrated. Financial transactions are simulated in memory.
4. **Dropped Negative Reputation Penalties:** Numerical deductions for task revision (-3), missed deadline (-5), abandoning a project midway (-20), and admin warnings (-15) defined in early concept drafts are **not implemented** in code. Warnings flag accounts and increment warning counters without decrementing reputation points.
5. **Absence of Early Completion Bonuses:** The system awards fixed XP based on difficulty (10/20/40 XP) and does not compute deadline deltas or early completion bonuses (+5 XP).
6. **Task File Uploads Excluded:** Binary file uploads are supported solely for user profile avatars (max 2MB); task deliverables are restricted to external URLs and notes.
7. **Direct Invitations & Kanban Boards Excluded:** Team formation is exclusively apply-only, and task tracking is strictly action-based.

## 19. Appendices

### Appendix A: Glossary of Terms

- **Active Project:** A project with status `Open` or `In Progress` that is not completed or cancelled.
- **Audit Entry:** An immutable log record capturing a critical system action, performing user, target entity, and timestamp.
- **Collaborator:** A user contributing technical effort by executing assigned project tasks.
- **Escrow Breakdown:** The canonical distribution of session fees: Owner funds $P + 0.15P + 0.02P$; Mentor receives $P$; Platform retains $0.15P$.
- **Idempotency Key / Flag:** The `xpAwarded` attribute ensuring task XP and reputation points are minted only once.
- **Maintenance Override:** System control state returning HTTP 503 to non-administrative users.
- **Mentor:** A vetted subject matter expert approved by an administrator to provide guidance and issue badges.
- **Project Owner:** The creator and team administrator of a project initiative.
- **Recommendation Badge:** A formal recognition token awarded by an assigned mentor granting +250 XP and +8 Reputation.
- **Super User:** The platform root administrator with unconstrained access to system configuration and admin provisioning.
- **XP:** Non-decreasing numerical measure of completed technical work.

### Appendix B: Requirement Mapping & Traceability Index

| Updated Req ID | Requirement Title | Category | Original ID | Traceability Status |
|---|---|---|---|---|
| **FR-01** | User Registration & Unique Identity | Authentication | FR1 | **CHANGED** |
| **FR-02** | Project Creation & Quota Enforcement | Projects | FR2 | **CHANGED** |
| **FR-03** | Apply-Only Project Recruitment | Projects | FR3 | **CHANGED** |
| **FR-04** | Task Management & Assignment Freezing | Tasks | FR4 | **CHANGED** |

| Updated Req ID | Requirement Title | Category | Original ID | Traceability Status |
|---|---|---|---|---|
| **FR-05** | Deliverable Evidence Submission | Tasks | FR5 | **CHANGED** |
| **FR-06** | Task Deliverable Review & Approval | Tasks | FR6 | **CHANGED** |
| **FR-07** | Automated XP Allocation by Difficulty | Gamification | FR7 | **CHANGED** |
| **FR-08** | Reputation Scoring & Inactivity Decay | Gamification | FR8 | **PARTIAL** |
| **FR-09** | Contribution Portfolio Reporting | Gamification | FR9 | **IMPLEMENTED** |
| **FR-10** | Multi-Period Leaderboard Generation | Gamification | FR10 | **IMPLEMENTED** |
| **FR-11** | Mentor Recommendation Badges | Mentorship | FR11 | **CHANGED** |
| **FR-12** | Multi-Tiered Administrative Governance | Admin | FR12 | **CHANGED** |
| **FR-13** | User Authentication & Suspension Check | Authentication | — | **NEW** |
| **FR-14** | Multi-Role Operational Switching | Identity | — | **NEW** |
| **FR-15** | Project Completion & Final URL Publication | Projects | — | **NEW** |
| **FR-16** | Project Member Removal for Cause | Projects | — | **NEW** |
| **FR-17** | Collaborator Voluntary Departure | Projects | — | **NEW** |
| **FR-18** | Unified Progress Calculation Engine | Projects / Tasks | — | **NEW** |
| **FR-19** | Mentor Application & Experience Tenure | Mentorship | — | **NEW** |
| **FR-20** | Mentor Marketplace Listing Profile | Mentorship | — | **NEW** |
| **FR-21** | Escrow-Backed Mentorship Booking | Escrow | — | **NEW** |
| **FR-22** | Escrow Release on Project Publication | Escrow | — | **NEW** |
| **FR-23** | Mentor Earnings & Admin Revenue Ledger | Escrow / Admin | — | **NEW** |
| **FR-24** | Project Workspace Team Chat Board | Communication | — | **NEW** |
| **FR-25** | Event-Driven User Notification Center | Notifications | — | **NEW** |
| **FR-26** | Platform Configuration Management | Configuration | — | **NEW** |
| **FR-27** | Maintenance Override Mode (HTTP 503) | Configuration | — | **NEW** |
| **FR-28** | User Profile & Theme Appearance Settings | Settings | — | **NEW** |
| **FR-29** | Help Desk & Support Request Management | Support | — | **NEW** |
| **FR-30** | Portal Admin Delegation & Permissions | Governance | — | **NEW** |
| **NFR-01** | Usability & Light/Dark Theme Switching | Usability | NFR1 | **IMPLEMENTED** |
| **NFR-02** | Header-Based RBAC Security Model | Security | NFR2 | **CHANGED** |
| **NFR-03** | Reliability, Idempotency & Fault Tolerance | Reliability | NFR3 | **IMPLEMENTED** |
| **NFR-04** | Performance & Sliding-Window Rate Limiting | Performance | NFR4 | **CHANGED** |
| **NFR-05** | In-Memory Volatile Storage Constraints | Architecture | — | **NEW** |